STAYEASE

Implementation Blueprint

Architecture, build sequence, APIs, data model, and verification plan

Version	1.1 — consolidated planning decisions
Status	Baseline for review before implementation
Delivery approach	Reference application → verification → trainee starter code
Scope target	Training-focused; approximately 5–8 days

Contents

1. Purpose and Implementation Rule
2. Proposed Repository Layout
3. Service Ownership
4. Important Data Entities (logical baseline)
5. API Contract Sketch
6. Inventory and Payment Design
7. Authentication Design
8. Failure and Resilience Behavior
9. Build Order
10. Test Plan and Verification Gate
11. Manual Infrastructure Runbook
12. Implementation Decisions to Resolve in Code-Ready Blueprint

1. Purpose and Implementation Rule

- This blueprint translates the consolidated StayEase SRS into a build sequence. The target is a complete reference application first, verified second, and trainee starter code only after verification.
- Do not claim that the app is 100% correct until automated and manual verification evidence exists. Track failures and limitations explicitly.

2. Proposed Repository Layout

stayease/ (root)

gateway-service/ (Spring Boot, port 8080)

user-service/ (Spring Boot, port 8081; stayease_user_db)

property-service/ (Spring Boot, port 8082; stayease_property_db)

reservation-service/ (Spring Boot, port 8083; stayease_reservation_db)

review-service/ (Spring Boot, port 8084; stayease_review_db)

stayease-ui/ (Angular 16.2.x, port 4200)

database/ (service-specific schema.sql and data.sql files)

docs/ (SRS, API contracts, data model, implementation plan, test plan, decisions, runbook)

postman/ (collection and environment templates)

README.md

- Each service should use layered packages: config, controller, dto, entity, repository, service, client, security, exception, audit, and mapper where needed. Avoid unnecessary abstractions that exceed the training scope.

3. Service Ownership

User Service: Users, password hashes, roles, tokenVersion, profiles, account audit records, login/register/email change/admin-assisted password reset.

Property Service: Hotels, destinations, amenities, room types, physical rooms, base rates, seasonal/weekend overrides, sellable capacity inputs, housekeeping status, property audit records.

Reservation Service: Reservations, inventory holds/reservations, reservation-price snapshots, simulated payment attempts, payment deadlines, cancellation/refund amounts, amendments, room assignments, invoice response, dashboard metrics, reservation audit records.

Review Service: Reviews, rating values, publication state/removal metadata, review audit records.

Gateway: Routing, common gateway config, CORS as needed, and gateway-level request handling. Do not make it a fifth business service.

4. Important Data Entities (logical baseline)

User Service: User(id, fullName, email unique, phone, address nullable, passwordHash, role USER/ADMIN, tokenVersion, createdAt, updatedAt); UserAuditEvent(id, actorId, action, entityId, eventType, timestamp, beforeJson, afterJson, description).

Property Service: Hotel(id, name, destination, address/contact fields, description, published rating derived from reviews or cached only with explicit rules); Amenity(id, name); HotelAmenity(hotelId, amenityId); RoomType(id, hotelId, name, description, maxGuests, baseNightlyRate, totalPhysicalRooms or sellableCapacity); PhysicalRoom(id, hotelId, roomTypeId, roomNumber unique within hotel, housekeepingStatus, operationalStatus); RateOverride(id, roomTypeId, date range, overrideType WEEKEND/SEASONAL, nightlyRate); PropertyAuditEvent(...).

Reservation Service: Reservation(id, publicReference, userId, hotelId, roomTypeId, checkInDate, checkOutDate, guestCount, reservationStatus, paymentPolicy PREPAID/PAY_LATER, paymentStatus, paymentDueAt, createdAt, confirmedAt, cancelledAt, cancellationReason, totalAmount, priceSnapshotJson or normalized nightly rows); PaymentTransaction(id, reservationId, idempotencyKey, method, amount, status, createdAt, completedAt, simulatedResult); RoomAssignment(id, reservationId, physicalRoomId, assignedAt, checkedInAt, checkedOutAt); Invoice is rendered from persisted booking/payment records rather than requiring a finalized invoice table; ReservationAuditEvent(...).

Review Service: Review(id, reservationId unique, userId, hotelId, rating, text, createdAt, status PUBLISHED/REMOVED, removedBy, removedAt, removalReason); ReviewAuditEvent(...).

Data modelling note: Cross-service identifiers are scalar IDs, not JPA cross-database relationships. Enforce ownership/eligibility via service APIs. Use database constraints within each service and DTO validation at service boundaries.

5. API Contract Sketch

User: POST /api/auth/register; POST /api/auth/login; GET /api/users/me; PUT /api/users/me; PATCH /api/users/me/email (currentPassword + newEmail); GET /api/admin/users?page=&size=&query=; POST /api/admin/users/{id}/password-reset.

Property: GET /api/hotels (paged filters); GET /api/hotels/{id}; GET /api/hotels/{id}/room-types; admin CRUD endpoints for hotels/amenities/room types/physical rooms/rate overrides; GET /api/room-types/{id}/availability?checkIn=&checkOut=&guests=; housekeeping status update endpoints.

Reservation: POST /api/reservations (select payment policy); GET /api/reservations/my?page=&size=; GET /api/reservations/{id}; POST /api/reservations/{id}/payment; POST /api/reservations/{id}/cancel; POST /api/reservations/{id}/amend; POST /api/reservations/{id}/check-in; POST /api/reservations/{id}/check-out; POST /api/admin/reservations/{id}/no-show; GET /api/reservations/{id}/invoice; GET /api/admin/dashboard.

Review: POST /api/reviews; GET /api/hotels/{hotelId}/reviews?page=&size=; DELETE or POST /api/admin/reviews/{id}/remove with reason; admin review list endpoints as required.

Audit: Each service exposes an admin-only paged audit endpoint with shared filters: from, to, category/eventType, actorId, entityId, page, size. No direct cross-database access.

Contract rule: These are proposed route shapes. Before coding, freeze request/response DTOs, status codes, validation errors, authentication requirements, idempotency semantics, and service-client contracts in docs/api-contract.md.

6. Inventory and Payment Design

- Room-type capacity is the booking/search abstraction. Use nightly availability over [checkIn, checkOut), with checkout date excluded from occupied nights.
- Prevent overbooking using a persisted reservation/hold ledger and transactionally consistent capacity checks within the authoritative inventory owner. Because cross-service ACID is unavailable, use an explicit hold/confirm/release protocol with idempotency and compensating actions.
- Prepaid flow: create expiring hold -> simulate payment with idempotency key -> if paid, confirm hold; if failed, release hold. A payment timeout enters an uncertain/reconciliation path; do not blindly retry or release capacity until status is resolved.
- Pay-later flow: reserve capacity and confirm -> paymentDueAt = min(createdAt + 24h, checkInDate at configured local check-in time) -> payment succeeds before deadline or scheduled expiry cancels and releases capacity idempotently.
- Booking availability and room assignment are separate: inventory can be secured for a future date even if no room is currently clean. At self-check-in, require a clean/eligible physical room with no overlapping assignment.
- Cancellation policy is fixed: >=24 hours before check-in is free; <24 hours retains one night's charge. Payment-timeout cancellation has no guest-initiated penalty. Use stored snapshot for charges/refunds.

7. Authentication Design

- Use Spring Security filter chain and JWT access token validation in each protected backend service (or a clearly documented shared library/config). Do not rely only on Gateway authentication.
- Token claims: subject/userId, role, tokenVersion, issued-at, expiration, issuer if configured. Store signing secret/key in environment/config outside committed source code.
- Each protected request compares tokenVersion claim to current User Service value. Define safe behavior when User Service is unavailable: protected critical operations fail closed. Use a bounded cache only if invalidation/TTL semantics are explicitly accepted; default is no cache for correctness.
- User registration always sets USER. Seed initial ADMIN by controlled SQL/setup. Do not commit a real/default production password; use an explicit local bootstrap mechanism and document manual password hash creation if SQL seed is used.
- Password reset increments tokenVersion. Email change requires current password, updates unique email, increments tokenVersion, and forces re-login. No email verification, lockout, rate limit, or refresh-token flow.

8. Failure and Resilience Behavior

- Use WebClient with service-discovery load balancing, bounded connect/read timeouts, normalized error DTOs, and Resilience4j where configured.
- Critical dependency unavailable: reject booking, payment, check-in/out, amendment, and review eligibility operation. Never confirm using stale data.
- Dashboard/audit may return partial results with per-source success/failure indicators and a visible frontend warning.
- Make cancellation, payment processing, inventory release, scheduled expiry, room assignment, and audit writes idempotent. Add correlation/request IDs to logs without secrets.

- Avoid circular call chains. Define one-directional contracts: Reservation calls Property for room/rate/room eligibility; Review calls Reservation for review eligibility; Property should not synchronously call Reservation to calculate availability if Reservation already calls Property. If search requires reservation counts, choose a dedicated one-way availability API and document the source of truth before implementation.

9. Build Order

Step 1 — Freeze contracts: Finalize entity/status enums, date/time policy, API DTOs, service-call direction, token validation strategy, and hold expiry. Commit docs before code.

Step 2 — Infrastructure configuration: Create service skeletons, Maven config, gateway/Consul configuration, environment property templates, and database scripts. Do not start MySQL or Consul automatically.

Step 3 — User Service: Registration, login/JWT, tokenVersion validation, profile, email change, admin user search/reset, user audit, unit/security tests.

Step 4 — Property Service: Hotels, room types, physical rooms, amenities, rate overrides, search filters, pagination, housekeeping, property audit, schema/seed scripts, tests.

Step 5 — Reservation core: Availability/hold protocol, price snapshot, prepaid and pay-later workflows, payment simulator, timeout scheduler, cancellation/refund, amendment, tests.

Step 6 — Stay operations and invoices: Self-check-in/out, physical room assignment, no-show admin operation, printable invoice API, dashboard metrics, reservation audit.

Step 7 — Review Service: Verified review eligibility, one review per reservation, 30-day limit, immediate publication, admin removal, review audit, tests.

Step 8 — Gateway and integration: Finalize Consul registration, routing, service clients, circuit breakers/timeouts, normalized errors, partial dashboard/audit aggregation.

Step 9 — Angular application: Auth forms, interceptor/guards, hotel search/filter/pagination, hotel details, reservation/checkout/payment, reservation management, self-check-in/out, invoice print view, reviews, admin dashboard, hotel operations, users, audit history.

Step 10 — Verification: Run tests after user manually initializes MySQL and starts Consul; test all acceptance scenarios, failure paths, permissions, SQL constraints, Postman workflows, UI flows, and SonarQube.

Step 11 — Trainee starter code: Only after the reference implementation passes the defined verification gate, create a separate starter branch/package with TODOs, tasks, acceptance criteria, and educator guide.

10. Test Plan and Verification Gate

Unit tests: Pricing precedence, tax/rounding, cancellation penalty, deadline calculation, status transitions, token-version mismatch, ownership rules, review eligibility, pagination filters.

Service integration tests: Repository constraints, DTO validation, exception mapping, transactions, idempotency, scheduled expiry, service-client error handling.

Security tests: Anonymous access rejected, invalid/expired token rejected, old token version rejected, USER cannot call ADMIN APIs, guest cannot read another user's reservation/invoice, public registration cannot assign ADMIN.

End-to-end workflows: Prepaid success/failure; pay-later paid/expired; cancellation/refund; amendment with increased/decreased total; check-in with clean room; check-in rejected when room unavailable; checkout makes room DIRTY; no-show; review; invoice; dashboard partial failure; audit filters.

Operational tests: MySQL schema/data scripts work from a clean DB; Consul registration works; Gateway routes to all services; service unavailability returns expected errors; manual startup instructions are accurate.

Verification gate: Do not start trainee starter-code generation until all critical acceptance scenarios pass, defects are triaged/fixed, tests and SonarQube results are recorded, and known limitations are documented.

11. Manual Infrastructure Runbook

- 1. User creates the four MySQL databases and manually executes each service's schema.sql and data.sql in the documented order.
- 2. User starts MySQL and verifies connections.
- 3. User starts Consul manually and verifies it is available on port 8500.
- 4. Start backend services and Gateway in the documented order, checking service registration and health.
- 5. Start Angular on port 4200.

- 6. Never let the agent start/stop MySQL or Consul. Whenever schema/data scripts are created or changed, notify the user to rerun initialization manually before dependent integration testing.

12. Implementation Decisions to Resolve in Code-Ready Blueprint

- Choose prepaid temporary-hold duration and expiration recovery semantics.
- Define inventory authority and hold API protocol precisely; avoid Property/Reservation circular calls.
- Choose check-in time-of-day and timezone policy for paymentDueAt when deadline is the scheduled check-in time.
- Choose tax default and rounding mode.
- Choose JWT expiry and local secret configuration; define how User Service tokenVersion checks are called by all services.
- Choose admin bootstrap/password hash process without committed default credentials.
- Choose audit aggregation mechanism and partial failure response schema.
- Document all selections in docs/decisions.md before dependent code is generated.